

Relatório do Projeto Final

Luiz Gustavo Turatti

Fase 2 – Processamento e Desempenho do EventMaster

1. Visão Geral e Objetivos da Arquitetura

Na Fase 2, o objetivo central é otimizar a arquitetura do EventMaster para garantir escalabilidade, processamento em tempo real, capacidades analíticas avançadas, alto desempenho do sistema, tolerância a falhas e consistência de dados em larga escala. A arquitetura evolui de uma simples "decomposição de serviços" para uma robusta plataforma distribuída orientada a dados. Essa evolução é fundamentada na adoção de processamento de fluxo (*stream*), processamento em lote (*batch*), arquitetura orientada a eventos, mensageria assíncrona e *pipelines* de dados distribuídos.

Estratégia do Modelo de Processamento

O ecossistema do EventMaster é estruturado sobre dois perfis distintos de processamento:

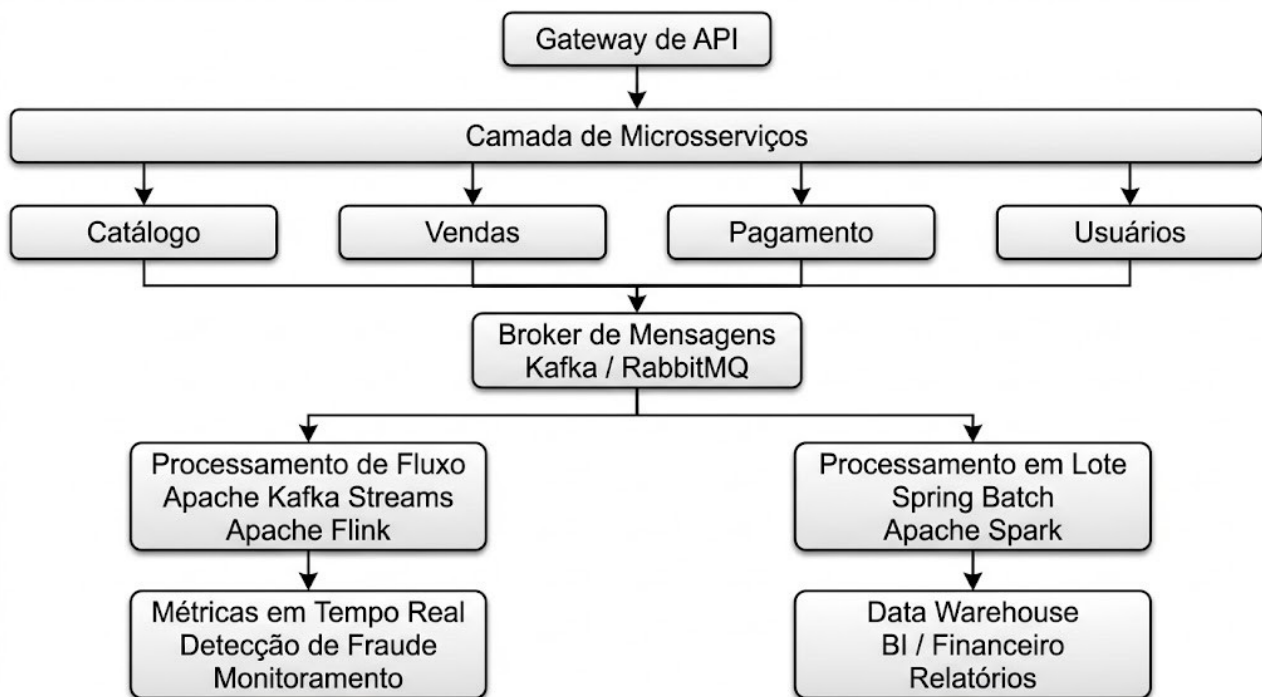
Tipo de Processamento	Características	Exemplos de Uso
Processamento em Fluxo (Stream)	Manipulação contínua de eventos em tempo real.	Gerenciamento de picos de acesso e detecção de fraudes.
Processamento em Lote (Batch)	Execução programada em larga escala.	Geração de relatórios diários e fechamento financeiro.

2. Arquitetura de Dados Proposta

O fluxo arquitetural foi redesenhado para suportar alta disponibilidade e assincronicidade. O tráfego de entrada passa pelo **API Gateway** e é direcionado à **Camada de Microsserviços** (composta por Catálogo, Vendas, Pagamentos e Usuários).

Esses serviços publicam eventos em um **Message Broker** central (como Kafka ou RabbitMQ) , que por sua vez alimenta duas ramificações de processamento paralelas:

- **Camada de Fluxo:** Utiliza Apache Kafka Streams ou Apache Flink para alimentar painéis de métricas em tempo real, sistemas de detecção de fraudes e monitoramento ativo.
- **Camada de Lote:** Emprega Spring Batch e Apache Spark para transferir dados consolidados para um *Data Warehouse*, gerando relatórios financeiros e de *Business Intelligence* (BI).



3. Processamento de Fluxo (*Stream*): Dados em Tempo Real

O principal objetivo desta camada é processar eventos de forma contínua com latência mínima.

Casos de Uso e Aplicação Prática

- **Monitoramento de Acessos em Tempo Real:** Grandes lançamentos de ingressos geram milhares de requisições simultâneas, atraindo tentativas de ataques DDoS, tráfego de bots e picos abruptos. A solução processa logs instantaneamente usando Kafka Streams ou Apache Flink. O fluxo transita do cliente para o API Gateway, entra no tópico Kafka access-events e é analisado pelo processador para gerar métricas de segurança.
- **Detecção de Fraudes:** O sistema identifica padrões anômalos, como tentativas excessivas de login, comportamentos de pagamento suspeitos, compras múltiplas originadas do mesmo IP e tráfego automatizado. Regras dinâmicas, como o bloqueio temporário de um IP após mais de 10 tentativas de login em um minuto, são aplicadas instantaneamente.
- **Gestão de Estoque e Pagamentos:** Durante picos de vendas, com milhares de reservas simultâneas, o estoque oscila a cada segundo; o processamento em fluxo garante que os *dashboards* reflitam essas mudanças instantaneamente. Além disso, instabilidades em provedores de pagamento são monitoradas de perto; se as falhas excederem a margem de 20%, o padrão *Circuit Breaker* é acionado preventivamente.
- **Painéis Analíticos (*Dashboards*):** Os organizadores têm visibilidade em tempo real sobre usuários ativos, receita gerada por minuto, saldo de ingressos e volume de transações simultâneas.

Pilha Tecnológica (*Stream*): Apache Kafka (*Event Broker*), Kafka Streams ou Apache Flink (*Stream Engine*) e Prometheus com Grafana (Monitoramento).

4. Processamento em Lote (*Batch*): Processamento Agendado

Dedicado ao processamento periódico de grandes volumes de dados. Diferentemente do modelo em fluxo, essa camada foca na agregação e estruturação de dados para fins analíticos e financeiros.

Casos de Uso e Aplicação Prática

- **Relatórios Financeiros e Auditoria:** Executados diariamente às 02h00 da manhã, os *jobs* geram relatórios detalhados para os organizadores, englobando ingressos vendidos, receita total, tributos aplicáveis, reembolsos processados e taxas operacionais. Adicionalmente, geram-se relatórios de conformidade imutáveis (logs de auditoria).
- **Inteligência de Negócios (BI) e Data Warehouse:** Os dados transacionais (OLTP) são extraídos dos microserviços e sincronizados com bases analíticas (OLAP). Esse repositório permite a consolidação de informações estratégicas, como eventos com melhor desempenho, distribuição regional de vendas, identificação de horários de pico e análise comportamental dos usuários.

Fluxo do Pipeline (*Batch*): Extração de dados do PostgreSQL -> Transformação -> Geração do Relatório Financeiro -> Armazenamento no *Data Warehouse* -> Notificação aos organizadores via e-mail. **Pilha Tecnológica (*Batch*):** Spring Batch (*Framework*), Apache Spark (ETL escalável), Quartz (*Scheduler*) e BigQuery ou Snowflake (*Data Warehouse*).

5. Arquitetura Orientada a Eventos e Resolução de Gargalos

A nova topologia transforma o EventMaster em um sistema reativo e orientado a eventos. O ciclo de vida do negócio é gerido por eventos centrais, incluindo criação de pedidos, *status* de pagamentos, e alocação ou liberação de estoque. Estes são orquestrados em tópicos Kafka dedicados (ex: access-events, order-events, payment-events).

Melhorias de Desempenho Implementadas:

- **Sobrecarga de Banco de Dados:** Mitigada pela adoção de bancos de dados distribuídos.
- **Picos de Requisições:** Absorvidos de forma eficiente através do mecanismo de *buffering* nativo do Kafka.
- **Lentidão em Relatórios:** Solucionada com a implementação paralela de processos de fluxo e *pipelines* em lote.
- **Instabilidade de Serviço e Inconsistência de Estoque:** Tratadas com a aplicação do padrão *Circuit Breaker* para resiliência e do padrão SAGA para garantir a integridade das transações distribuídas.
- **Impacto Analítico no Ambiente de Produção:** Isolado por meio do envio de cargas de trabalho pesadas para um *Data Warehouse* dedicado.

6. Segurança e *Clean Architecture*

As práticas de segurança foram desenhadas para abranger as especificidades de cada modelo de processamento. A camada de fluxo realiza o monitoramento ativo de anomalias (IPs suspeitos, abuso de tokens e ataques de força bruta). Simultaneamente, a camada em lote assegura o tráfego e armazenamento de dados por meio de relatórios financeiros criptografados, *pipelines* de ETL seguros e controles de acesso rigorosos para dados de BI.

A *Clean Architecture* foi aplicada para garantir a separação de responsabilidades. A camada de fluxo isola produtores, tópicos Kafka, consumidores e análises em tempo real, enquanto a camada de lote gerencia de forma independente os agendadores, *jobs* ETL e relatórios do *Data Warehouse*.

Nota de Implementação: O projeto inclui protótipos em Java validados. Para a arquitetura de fluxo, o monitoramento em tempo real rastreia a volumetria de acesso por IP e emite alertas automatizados em caso de latência superior a 1000 milissegundos ou indícios de tráfego de bots. Para a arquitetura em lote, foram consolidados serviços capazes de iterar e somar faturamentos e volumes de vendas de múltiplos eventos programaticamente.

7. Conclusão da Evolução Arquitetural

A Fase 2 representa a maturação técnica do EventMaster. A arquitetura progrediu da conversão inicial de monólito para microsserviços (Fase 1) para um ecossistema complexo que integra *Event-Driven Architecture*, processamento em tempo real e análises distribuídas em lote.

Através da combinação sinérgica de microsserviços, *streaming* via Kafka, *pipelines* ETL, *Clean Architecture*, *Domain-Driven Design* (DDD), orquestração SAGA e *Circuit Breakers*, a plataforma suporta lançamentos de ingressos em escala massiva. Esta evolução consolida o EventMaster como uma plataforma corporativa *cloud-native* verdadeiramente escalável, segura e observável, pronta para sustentar altos volumes de transações com excepcional confiabilidade.